

CHẤT LƯỢNG BẢN THIẾT KẾ

(DESIGN QUALITIES)

Chất lượng phần mềm

- ☕ Sự khác biệt về **tính hữu hình**:
 - 🌿 **Trong sản xuất truyền thống**: Chất lượng là thứ có thể cầm nắm và quan sát trực quan như độ mịn của nước sơn xe hay độ khít của các khớp gỗ.
 - 🌿 **Trong phần mềm**: Chất lượng mang tính trừu tượng cao, nằm sâu trong **cấu trúc logic** và các lớp **kiến trúc** bên dưới.

Chất lượng phần mềm

Hệ quả của **sự vô hình**:

- ☕ Người dùng thường bị đánh lừa bởi vẻ ngoài của **giao diện**.
 - 🌿 Một ứng dụng ngân hàng có màu sắc thời thượng nhưng xử lý giao dịch mất nhiều thời gian hoặc thường xuyên vắng ứng dụng thì vẫn bị coi là sản phẩm kém chất lượng.
- ☕ Chất lượng thực sự phải là *sự tổng hòa giữa trải nghiệm người dùng và sự vững chắc của hệ thống*.

Thách thức trong việc Đánh giá

- ☕ Ranh giới giữa **Thiết kế** và **Cài đặt**:
 - 🌿 Trong xây dựng, bản vẽ và tòa nhà là hai thực thể tách biệt.
 - 🌿 Trong phần mềm, **mã nguồn chính là bản thiết kế chi tiết nhất**, sai lầm trong tư duy sẽ dẫn đến *sai lệch cài đặt* ngay lập tức.
- ☕ Đánh giá **đa chiều** để có cái nhìn toàn diện:
 - 🌿 Kiểm tra **Tĩnh**: Tập trung soi xét *cấu trúc mã*, tính dễ đọc và việc tuân thủ các tiêu chuẩn lập trình.
 - 🌿 Kiểm tra **Động**: Quan sát phần mềm khi vận hành để đo lường *tốc độ xử lý, mức chiếm dụng bộ nhớ và khả năng phục hồi*.

Fitness for purpose

- ☕ **Mục tiêu tối thượng** của chất lượng:
 - 🌿 Chất lượng không phải là sự hoàn hảo mà là việc phần mềm **làm tốt nhất nhiệm vụ** được giao.
 - 🌿 Một công cụ tốt phải giải quyết đúng **nỗi đau của người dùng** trong kịch bản thực tế.
- ☕ Khả năng **thích nghi và cân bằng**:
 - 🌿 Thiết kế xuất sắc phải biết cân bằng giữa các ràng buộc về **ngân sách, thời gian và phần cứng**.
 - 🌿 Phần mềm chỉ thực sự giá trị khi vận hành ổn định trên **thiết bị của khách hàng** chứ không phải chỉ chạy tốt trên **máy tính của lập trình viên**.
- ☕ **Sự phù hợp** luôn quan trọng hơn sự hào nhoáng.

ĐÁNH GIÁ CHẤT LƯỢNG THIẾT KẾ

Nếu bạn không thể đo lường và biểu diễn nó bằng con số, sự hiểu biết của bạn là ít ỏi và không thỏa đáng - *Lord Kelvin*

- ☕ Khó khăn đặc thù trong ngành phần mềm:
 - 🌿 Chúng ta thiếu một **thước đo vật lý** chính xác để đo trực tiếp *vẻ đẹp* hay *độ tinh tế* của một kiến trúc.
 - 🌿 Các khái niệm như **tính linh hoạt** thường mang tính trừu tượng và khó quy đổi ra đơn vị đo lường chuẩn.

Các hệ thống thang đo trong đánh giá

- ☕ **Thang đo thứ bậc** (Ordinal Scale):
 - 🌿 So sánh mức độ giữa các đối tượng.
 - 🌿 VD: **Thiết kế A dễ bảo trì hơn thiết kế B** hoặc *Kiến trúc Microservices linh hoạt hơn Monolith.*
 - 🌿 Thang đo này giúp ta **xếp hạng ưu tiên** nhưng chưa chỉ ra được khoảng cách chính xác giữa các lựa chọn.
- ☕ Rào cản:
 - 🌿 Trong thực tế, rất khó để đưa ra các con số tuyệt đối như: *Mã nguồn này tốt gấp 2.5 lần mã nguồn kia.*
 - 🌿 Sự phức tạp của phần mềm không tăng trưởng tuyến tính, khiến việc thiết lập một **tỷ lệ toán học** là rất khó.

Đánh giá Tĩnh (Static Evaluation)

- ☕ Soi xét mã nguồn khi chương trình *đang ở trạng thái nghỉ*, chưa được thực thi.
 - 🌿 Giúp phát hiện sớm các lỗi về logic, bảo mật và vi phạm tiêu chuẩn lập trình ngay từ giai đoạn viết code.
- ☕ Các tiêu chí kiểm tra điển hình:
 - 🌿 **Sự tuân thủ chuẩn mực**: Kiểm tra quy tắc đặt tên, cách thụt lề và cấu trúc file để đảm bảo tính đồng nhất trong toàn dự án.
 - 🌿 **Nguyên lý DRY** (Don't Repeat Yourself): Phát hiện các đoạn mã bị lặp lại, giúp giảm thiểu công sức bảo trì và nguy cơ sai sót khi cập nhật.
 - 🌿 **Độ phức tạp của mã**: Đo lường xem các hàm có quá dài hay các vòng lặp có quá chồng chéo hay không.

Đánh giá Động (Dynamic Evaluation)

- ☕ Quan sát và đo lường hệ thống khi nó *đang vận hành thực tế* trong môi trường giả lập hoặc thực tế.
- ☕ Tập trung vào các yếu tố mà việc đọc mã nguồn đơn thuần không thể tiên lượng hết được.
- ☕ Các kịch bản kiểm tra quan trọng:
 - 🌿 **Hiệu năng dưới áp lực:** Đo lường thời gian phản hồi khi hệ thống đối mặt với **tải trọng lớn**, ví dụ như có 10.000 người dùng truy cập cùng lúc.
 - 🌿 **Quản lý tài nguyên:** Theo dõi mức độ chiếm dụng CPU và kiểm tra các lỗi **rò rỉ bộ nhớ** (memory leak) có thể làm sập hệ thống sau một thời gian dài hoạt động.
 - 🌿 **Trải nghiệm thực tế:** Đảm bảo các luồng nghiệp vụ chạy đúng và mượt mà trong điều kiện mạng hoặc phần cứng hạn chế.

CÁC "ILITIES" (CÁC ĐẶC TÍNH CHẤT LƯỢNG)

- 🍷 Đây là những thuộc tính kết thúc bằng hậu tố *ility*, đại diện cho những khía cạnh phi chức năng cốt lõi của phần mềm.
- 🍷 Chất lượng không phải là một hằng số cố định mà là một **sự đánh đổi** (trade-off) liên tục giữa các yếu tố khác nhau.
 - 🌿 **Độ tin cậy (Reliability):** Hệ thống có chạy đúng và ổn định không?
 - 🌿 **Hiệu suất (Efficiency):** Hệ thống có tốn tài nguyên không?
 - 🌿 **Khả năng bảo trì (Maintainability):** Sửa lỗi và nâng cấp có dễ không?
 - 🌿 **Tính khả dụng (Usability):** Người dùng có dễ sử dụng không?

Độ tin cậy (Reliability)

Đảm bảo hệ thống hoạt động ổn định trong thời gian dài.

- ☕ **Tính đầy đủ (Complete):** Không bỏ sót trường hợp ngoại lệ (Edge cases).
 - 🌿 **VD:** App thanh toán phải xử lý được cả khi mạng yếu, hết tiền, hoặc server ngân hàng bảo trì.
- ☕ **Tính nhất quán (Consistent):** Cùng một đầu vào phải cho ra cùng một kết quả.
- ☕ **Khả năng phục hồi (Resilient):** Hệ thống không sụp đổ hoàn toàn khi gặp sự cố cục bộ (**Graceful Degradation**).
 - 🌿 Thay vì ngừng hoạt động, hệ thống tự động ngắt bỏ các tính năng phụ để duy trì luồng nghiệp vụ cốt lõi.
 - 🌿 **VD:** Netflix sử dụng **Chaos Monkey** để tự đánh sập server của chính mình nhằm kiểm tra khả năng tự phục hồi của hệ thống.



Hiệu suất (Efficiency)

Tối ưu hóa việc sử dụng tài nguyên hệ thống.

- ☕ **Các nguồn lực:** CPU, RAM, Băng thông, Dung lượng lưu trữ.
- ☕ **Sự đánh đổi (Trade-offs):**
 - 🌿 **Ví dụ:** Sử dụng **Caching** giúp ứng dụng chạy nhanh hơn (tăng hiệu suất thời gian) nhưng lại tốn RAM hơn (giảm hiệu suất bộ nhớ).
 - 🌿 Một đoạn code viết quá tối ưu bằng hợp ngữ có thể chạy cực nhanh nhưng lại cực kỳ khó bảo trì.
- ☕ **Mục tiêu:** Đạt được hiệu năng đủ tốt trong mức chi phí cho phép.



Các khía cạnh Kỹ thuật & Nhận thức (HCI Dimensions)

- ☕ **Cam kết vội vàng (Impetuous Commitment):** Xảy ra khi một quyết định thiết kế được đưa ra quá sớm khi chưa có đủ thông tin
 - 🌿 Chốt sử dụng một loại database cụ thể (như MongoDB) khi chưa xác định rõ cấu trúc và mối quan hệ dữ liệu, dẫn đến khó khăn khi cần thực hiện các truy vấn phức tạp về sau.
- ☕ **Phụ thuộc ẩn (Hidden Dependencies):** Mối liên kết giữa hai thành phần không được hiển thị rõ ràng thông qua ký pháp hoặc sơ đồ
 - 🌿 Một thay đổi nhỏ tại Module A có thể làm sụp đổ Module B.
 - 🌿 Đòi hỏi người bảo trì phải có **kiến thức ngầm** quá lớn, dễ gây ra lỗi nghiêm trọng khi nhân sự thay đổi.

Các khía cạnh Kỹ thuật & Nhận thức (HCI Dimensions)

- ☕ **Ký pháp phụ (Secondary Notation):** Tận dụng các yếu tố phi kỹ thuật như cách thụt lề, comment, hoặc cách đặt tên để truyền tải ý đồ thiết kế.
 - 🌿 Cách thụt lề code (**Indentation**), cách đặt tên biến (**Naming conventions**).
 - 🌿 Sử dụng ghi chú (**Comments**) hoặc cách sắp xếp các khối sơ đồ cạnh nhau để thể hiện sự liên quan.
- ☕ **Độ nhớt (Viscosity):** Chỉ số đo lường sự **kháng cự thay đổi** của hệ thống. Độ nhớt càng cao, việc chỉnh sửa càng tốn kém và dễ sai sót
 - 🌿 **VD:** Thay vì dùng hằng số `TAX_RATE = 0.1`, bạn viết trực tiếp số `0.1` vào 100 file khác nhau. Khi thuế thay đổi, bạn phải sửa 100 chỗ. Đó là **Độ nhớt cao**.

ĐẶC ĐIỂM CỦA SẢN PHẨM THIẾT KẾ TỐT

Sự đơn giản (Simplicity)

Nguyên lý Einstein: Mọi thứ nên được làm đơn giản nhất có thể, nhưng không được đơn giản hơn thế.

- ☕ **KISS (Keep It Simple, Stupid):** Đừng cố phức tạp hóa vấn đề bằng những thiết kế thừa thãi (Over-engineering).
- ☕ **Các thước đo độ phức tạp:**
 - 🌿 **McCabe:** Đo số lượng các nhánh rẽ (if/else).
 - 🌿 **Henry & Kafura:** Đo lượng thông tin luân chuyển giữa các module.
- ☕ **VD:** Một hàm dài 500 dòng với 20 câu lệnh `if` lồng nhau là kẻ thù của sự đơn giản.



Tính Mô-đun (Modularity)

Chia hệ thống lớn thành các mảnh nhỏ (**Lego-like**).

- ☕ **Kết dính (Cohesion) - Cao là tốt:** Mỗi module chỉ làm duy nhất một việc và làm thật tốt.
 - 🌿 **VD:** Class `Chef` chỉ nấu ăn, không nên kiêm luôn việc `Accounting` (kế toán).
- ☕ **Kết ghép (Coupling) - Thấp là tốt:** Các module ít phụ thuộc vào nhau nhất có thể.
 - 🌿 **VD:** Thay đổi công thức nấu ăn của `Chef` không nên làm hỏng phần mềm `Accounting`.
- ☕ **Dấu hiệu xấu: Switch Coupling** - Khi Module A gửi một chiếc cờ (flag) để ra lệnh cho Module B phải chạy bước nào.
 - 🌿 Module A đang can thiệp quá sâu vào logic nội bộ của Module B.
 - 🌿 Đây là dấu hiệu của một thiết kế lỗi, khi một module cố gắng làm quá nhiều việc khác nhau tùy theo **lệnh** từ bên ngoài.



Che giấu thông tin (Information Hiding)

- ☕ **Nguyên lý Parnas:** Che giấu các quyết định thiết kế dễ thay đổi đằng sau các giao diện ổn định.
- ☕ **Bức tường phạm vi (Scope Wall):** Bên ngoài chỉ cần biết module làm được gì, không cần biết nó làm như thế nào.
- ☕ **Ví dụ về cấu trúc Stack:**
 - 🌿 Người dùng chỉ gọi `push()` và `pop()`.
 - 🌿 Bên trong dùng **Mảng** hay **Danh sách liên kết** là bí mật của module. Nếu sau này ta đổi từ mảng sang danh sách, các code bên ngoài không cần sửa một dòng nào.



Các đặc điểm tiêu cực cần tránh

- ☕ **State Data phân tán:** Trạng thái của ứng dụng (ví dụ: User đang login hay chưa) bị lưu rải rác ở nhiều biến toàn cục.
- ☕ **FAT Interface:** Hàm có quá nhiều tham số (ví dụ: hàm `draw()` nhận vào 20 tham số thay vì 1 object `config`).
- ☕ **Control Coupling phức tạp:** Luồng điều khiển nhảy lung tung giữa các module.
- ☕ **Duplicate Code:** Lặp lại logic xử lý ở nhiều nơi (vi phạm **DRY**).
- ☕ **Mô-đun yếu:** Một class `Utility` chứa đủ thứ hàm không liên quan đến nhau.

ĐÁNH GIÁ QUY TRÌNH THIẾT KẾ

Quy trình tốt sẽ tạo ra sản phẩm tốt một cách bền vững.

Mô hình **CMM** (Capability Maturity Model)

- 1 Initial:** Làm việc kiểu **anh hùng**, mỗi người một kiểu, thành bại do cá nhân.
- 2 Repeated:** Bắt đầu có quy trình quản lý, dự án sau có thể lặp lại thành công của dự án trước.
- 3 Defined:** Có bộ quy chuẩn chung cho cả công ty.
- 4 Managed:** Mọi thứ được đo bằng con số (VD: mật độ lỗi/1000 dòng code).
- 5 Optimizing:** Luôn tìm cách cải tiến để quy trình ngày càng hoàn thiện hơn.



Phản hồi để cải thiện quy trình

Đừng đợi đến khi sản phẩm hoàn tất mới kiểm tra chất lượng.

- 1 Đánh giá kỹ thuật (Technical Reviews):** Đồng nghiệp ngồi lại soi lỗi thiết kế của nhau (**Code Review** / **Design Review**). Đây là cách rẻ nhất để tìm lỗi.
- 2 Đánh giá quản lý:** Nhìn lại xem tiến độ có khớp không, tại sao một module lại tốn thời gian hơn dự kiến?
- 3 Tạo mẫu (Prototyping):** Làm bản demo nhanh để khách hàng dùng thử.
 -  **Ví dụ:** Làm mẫu giao diện bằng Figma để khách chốt luồng đi trước khi lập trình viên bắt đầu gõ code.



THIẾT KẾ TRONG KỶ NGUYÊN AGENTIC CODING

Khi AI không chỉ là công cụ gợi ý mà là **Agent trực tiếp thực thi**, chất lượng thiết kế cần hướng tới sự *thấu hiểu* của cả con người và máy.



THIẾT KẾ TRONG KỸ NGUYÊN AGENTIC CODING

AI-Reasonability (Khả năng suy luận của AI):

-  Thiết kế phải đủ rõ ràng để AI Agent có thể nắm bắt ý đồ thiết kế mà không cần đọc toàn bộ codebase.
-  Hạn chế các *phép thuật* (Magic code) hoặc cấu trúc quá phức tạp mà AI dễ gây ra ảo-giác (**Hallucination**).

Test-Driven Design:

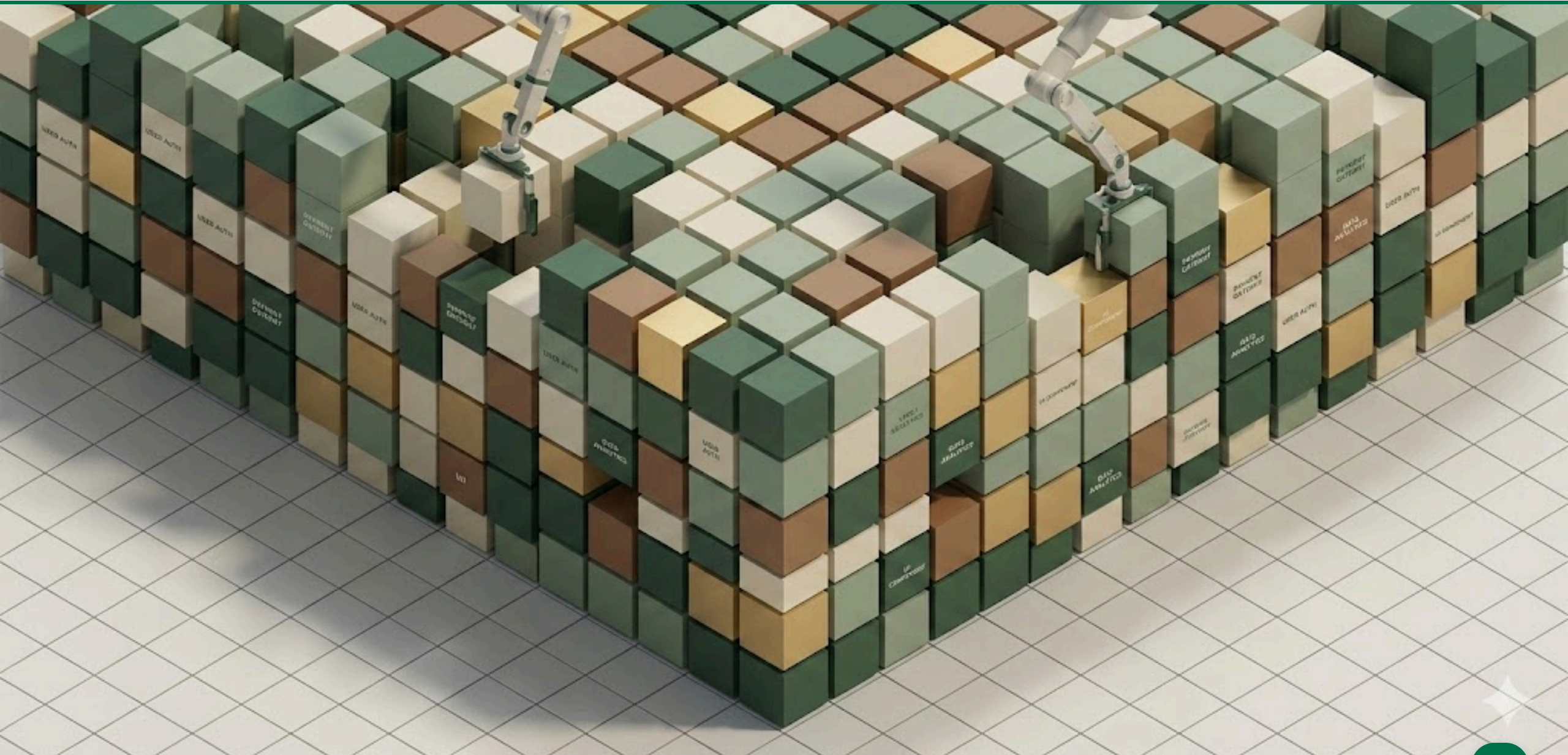
-  **Testability** trở thành ưu tiên sống còn. AI Agent cần một *hệ thống phản hồi* (**Feedback Loop**) thông qua **Unit Test** để tự sửa lỗi.
-  Nếu mã nguồn khó viết Test, AI Agent sẽ trở nên nguy hiểm thay vì hữu ích.



CONTINUOUS IMPROVEMENT CYCLE

THIẾT KẾ TRONG KỶ NGUYÊN AGENTIC CODING

- ☕ **Contextability (Khả năng đóng gói ngữ cảnh):**
 - 🌿 Do giới hạn về **Context Window** (cửa sổ ngữ cảnh) của LLM, hệ thống cần có tính **Modularity** cực cao.
 - 🌿 Mỗi Module/File nên đủ nhỏ và độc lập để AI có thể xử lý trọn vẹn trong một lượt làm việc mà không bị "quên" logic.
- ☕ **Standardization over Innovation:**
 - 🌿 Ưu tiên sử dụng các Design Pattern chuẩn mực và phổ biến.
 - 🌿 Sự nhất quán (**Consistency**) giúp AI dự đoán và tuân thủ phong cách lập trình của dự án tốt hơn, giảm thiểu sai sót khi Refactoring tự động.



TỔNG KẾT

- ☕ **Hai mặt của chất lượng:** Cấu trúc tĩnh (Code đẹp) và Hành vi động (Chạy tốt).
- ☕ **Fitness for Purpose:** Là thước đo cuối cùng
 - 🌿 **Chức năng là ưu tiên số 1:** Một chiếc ghế dù đẹp đến đâu mà không ngồi được thì vẫn là một thiết kế thất bại.
 - 🌿 **Tính thực tế:** Code đẹp hay kỹ thuật cao siêu đều trở nên vô nghĩa nếu không giải quyết được vấn đề thực tế của người dùng.
 - 🌿 Thiết kế cần bám sát phân tích yêu cầu để đảm bảo sự phù hợp.
- ☕ **Trừu tượng hóa:** Giúp chúng ta quản lý độ phức tạp nhưng làm việc đo lường khó khăn hơn.
- ☕ **Review & Prototype:** Là công cụ đặc lực để phát hiện sai sót từ sớm.

CÂU HỎI ÔN TẬP

- 1 Tại sao nói đánh giá chất lượng phần mềm khó hơn đánh giá cái bàn, cái ghế?
- 2 Cho một ví dụ thực tế về **Reliability** trong ứng dụng Grab/Be.

Q&A

CẢM ƠN ĐÃ THEO DÕI